

Practice : Max-Flow

Deadline: June.3/4/5. Must be checked onsite (during labs).Online checks via email or messaging apps are not accepted.

1. Objective

This Practice has one question based on **Chapter 7 part1 and part2 Max-Flow** in the lecture. You need understand the application of the **Ford-Fulkerson** algorithm and **Edmonds-Karp** (Shortest augmenting path in Lecture) algorithm.

Other than that, you need to be able to clearly explain the **differences** between the two algorithms and their **time complexities**.

2. Task

The **Ford-Fulkerson algorithm** is the fundamental method for solving the maximum flow problem.

Description:

Given n nodes, m edges, and the weight of each edge is larger than 0. Find the max-flow of the graph. In the graph, the node 0 is the source node and the node $n-1$ is the sink node.

Input Description:

The first line contains two integers **n** and **m**, representing n nodes and m edges, separated by a space.

The next **m** lines each contain three integers: the first two integers represent the node indices, and the third integer represents the **weight** between the two nodes.

Output Description:

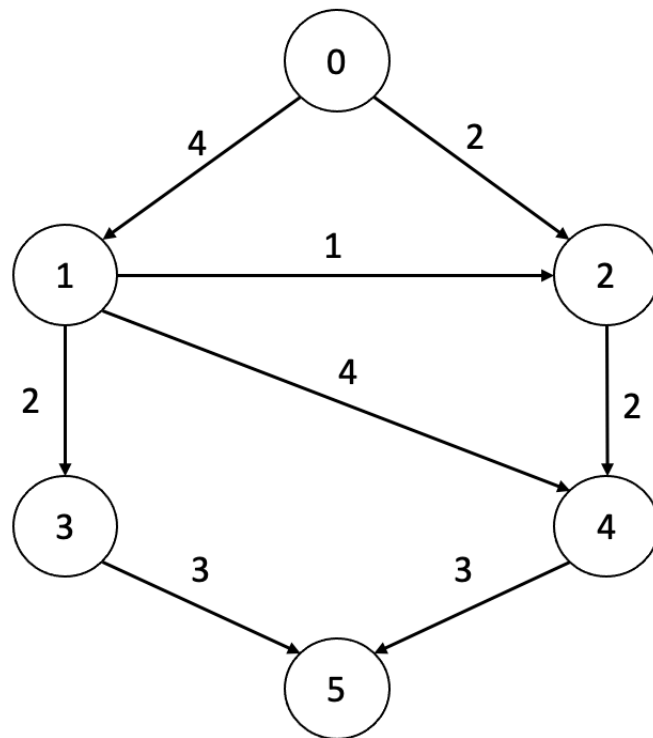
Output the max-flow of the graph.

Input Test case1:

```
6 8
0 1 4
0 2 2
1 2 1
1 4 4
2 4 2
4 5 3
1 3 2
3 5 3
```

Output1

5

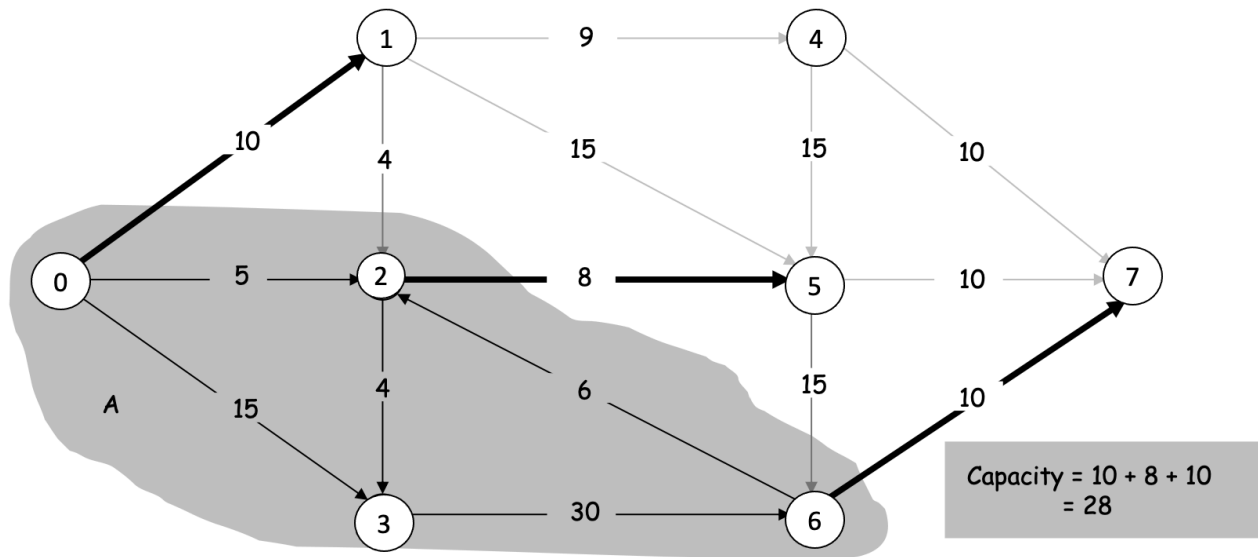


Input Test case2:

```
8 15
0 1 10
0 2 5
0 3 15
1 2 4
2 3 4
1 4 9
1 5 15
2 5 8
3 6 30
6 2 6
4 5 15
5 6 15
4 7 10
5 7 10
6 7 10
```

Output2

28



Input Test case3:

In the file `1.in`

Output Test case3:

172

3. Introduction

3.1 Basic Concepts

Augmenting Path (增广路): A simple path from the source node `s` to the sink node `t` in the residual graph. The edges in the residual graph represent the potential to "increase flow further," including forward edges (which can still push more flow) and backward edges (which can cancel/return flow).

Bottleneck Capacity (瓶颈容量): The minimum residual capacity among all edges along an augmenting path. This is the maximum amount of flow that can be added along this path — once any edge is saturated, the entire path can no longer push additional flow.

Augment Function: The operation that updates flows along an augmenting path:

- **Forward edge** (an edge present in the original graph): $flow- = bottleneckCapacity$
- **Backward edge** (an edge in the residual graph used to cancel/return flow): $flow+ = bottleneckCapacity$

3.2 Algorithm Process

1. Create a **residual graph**.
2. As long as there exists an **augmenting path(增广路)** in the residual graph:
 - Find an **augmenting path**
 - Identify the bottleneck capacity `x` along the path

- Decrease the residual capacity of each edge along the path by x . Remove edges where residual capacity becomes zero.
 - Add a backward (backward) edge with capacity x
3. When no path from the source to the sink exists, the maximum flow of the graph is: the original capacity of **out-degree(出度) edges** from the source node **minus** the remaining residual capacity of those same **out-degree(出度) edges**.

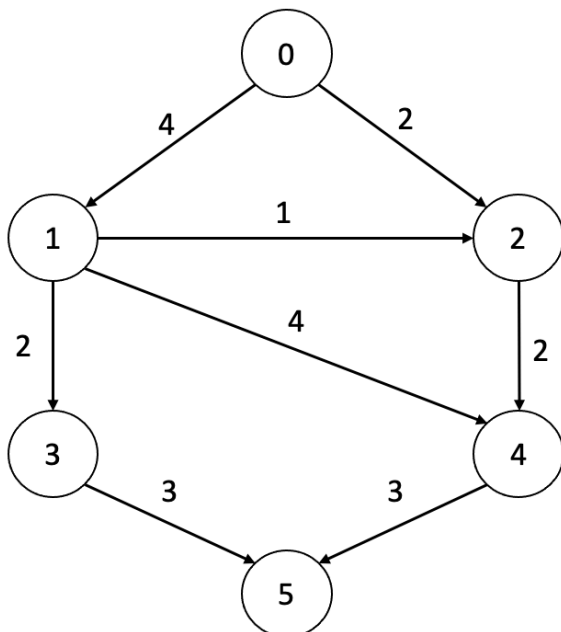
3.3 Meaning of Backward Edges

The essence of backward edges: they do not represent actual reverse flow, but rather a "mechanism for undoing/revising previous flow decisions"

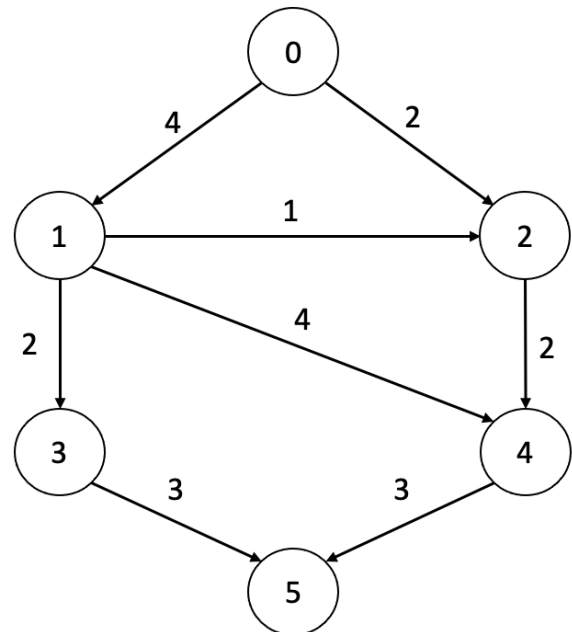
3.4 Test1 Sample Demonstration

Original Graph:

Original Graph

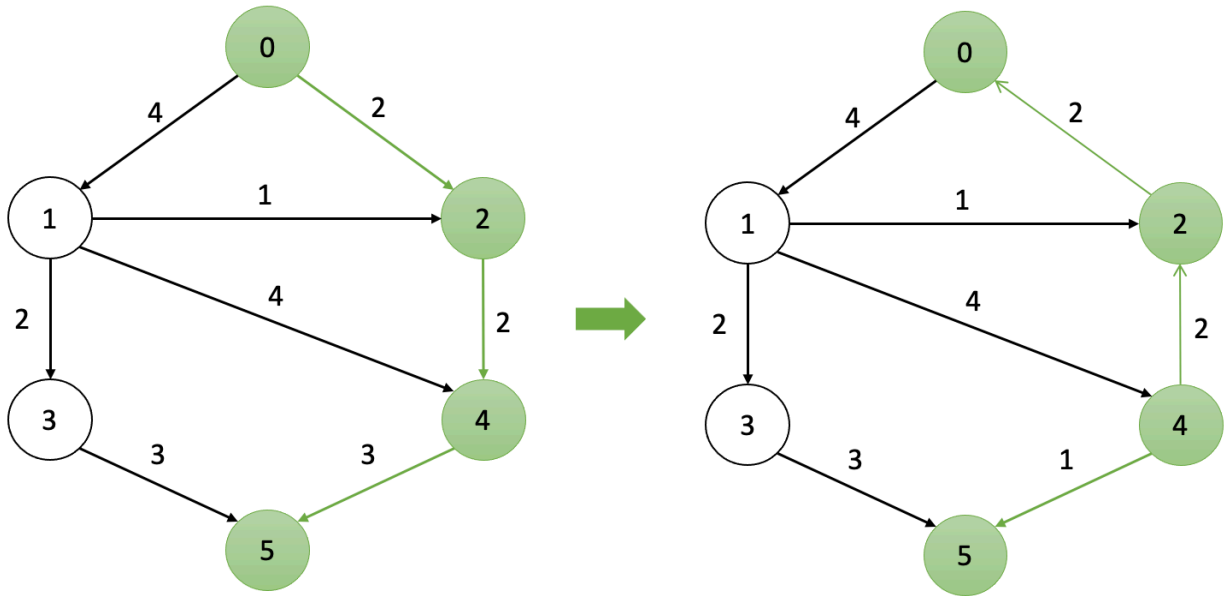


Residual Graph



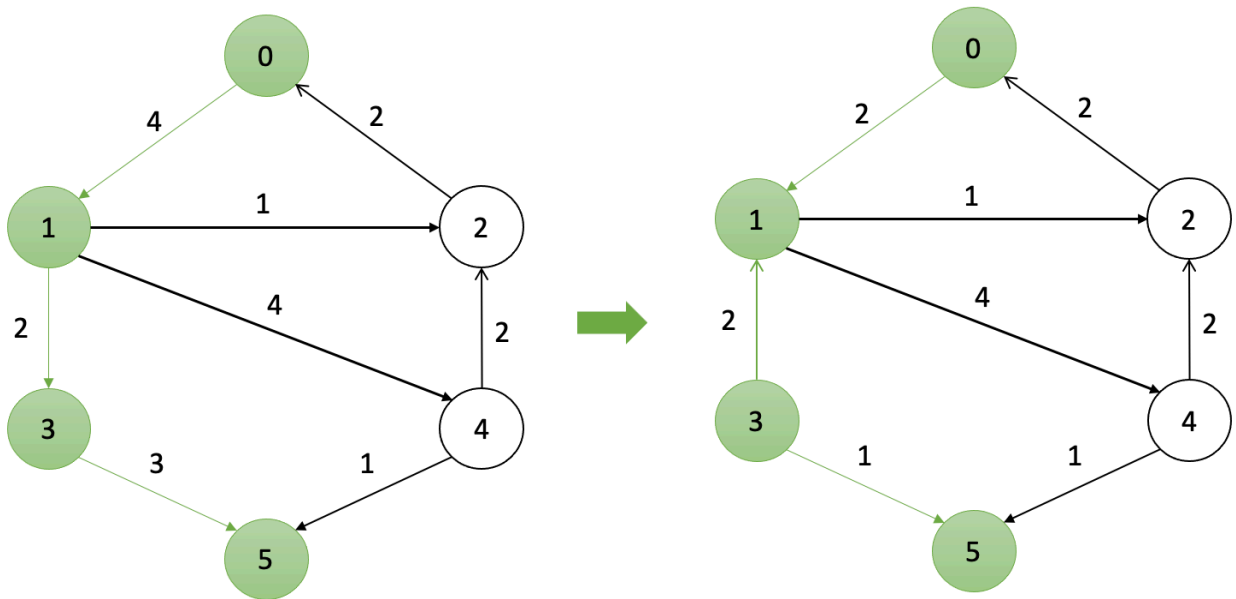
Find augmenting path(增广路): **0->2(2) --- 2->4(2) --- 4->5(3)**

Bottleneck Capacity (瓶颈容量) = 2



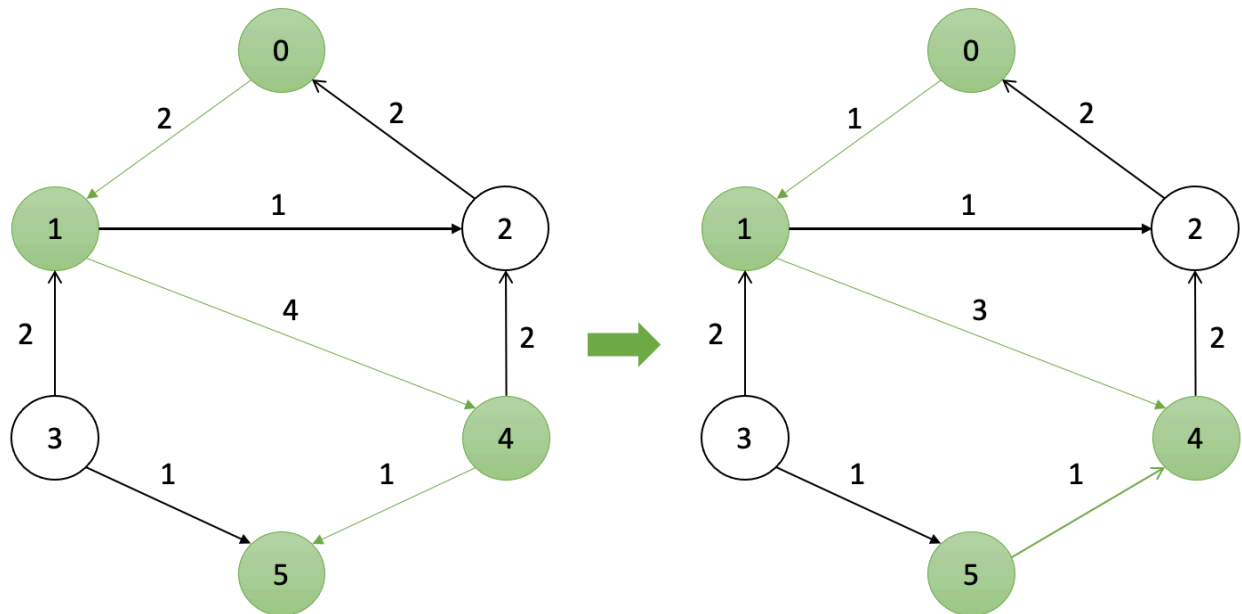
Find augmenting path(增广路): 0->1(4) --- 1->3(2) --- 3->5(3)

Bottleneck Capacity (瓶颈容量) = 2



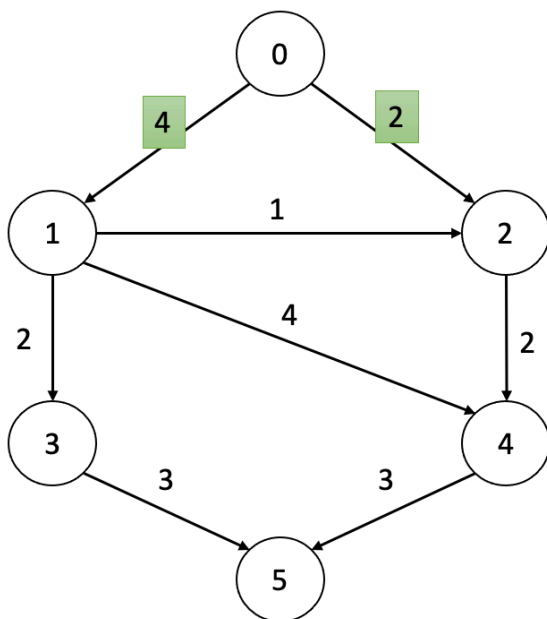
Find augmenting path(增广路): 0->1(2) --- 1->4(4) --- 4->5(1)

Bottleneck Capacity (瓶颈容量) = 1

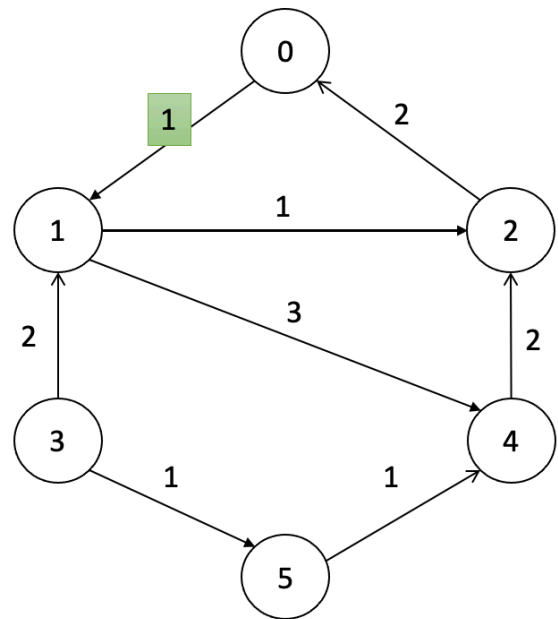


Difference between the out-degree edge in original graph and the residual graph is 5, so the **max-flow** and **minimum cut** is 5.

Original Graph



Residual Graph



3.5 Edmonds-Karp (Shortest augmenting path in Lecture)

When searching for an **augmenting path**, using **breadth-first traversal** — i.e., giving priority to the path with the fewest edges — results in the **Edmonds-Karp** algorithm.

4. Submission

Show the following to your tutor or SA **onsite**:

Grading:

- Submissions on time:

- 0.5 points for first two test cases.
- 0.3 points test case 3.
- 0.2 for answer the question: in which case, the **Edmonds-Karp** (Shortest augmenting path in Lecture) is more faster than **Ford-Fulkerson** algorithm.
- Late submissions: 0 points.